Scope

Testing will cover both applications and their shared SQLite database.

- Python Employee App: Login, expense submission, viewing, editing, deletion, and review history.

- Java Manager App: Login, pending expense listing, approving, denying, reports, and CSV export.

- Shared Database: Database constraints, data consistency, expense status transitions, and cross-application functionality.

- Validation: Input validation, authorization, logging, and error handling.

Testing Approach

- Unit Testing: Use PyTest for Python functions and services, and JUnit 5 with Mockito for Java methods and services.
- Integration Testing: Test application services against real temporary SQLite databases.
- SQL Testing: Execute five SQL validation queries to verify database integrity. Each query must return zero rows.
- Menu Testing: Verify login, menu navigation, valid choices, invalid choices, and application exit.
- Shared validation test data: Both apps check expense IDs, menu choices, and empty login input using data from the same file.
- Cross-Application Testing: Manually verify all six scenarios to confirm that employee submissions and manager decisions are correctly shared.
- Exploratory Testing: Conduct two 30-minute sessions, one for each application, to discover unexpected defects.
- Regression Testing: Rerun automated tests and previously failed test cases after defect fixes.

Test design techniques will include Equivalence Partitioning, Boundary Value Analysis, Decision Tables, and State Transition Testing.

Entry Criteria

- Both applications are available for testing.
- Database schema and seed data are prepared.
- Python, Java, and required dependencies are installed.
- Test cases and test data are documented.
- Test database connections are working.

Exit Criteria

- At least 30 designed test cases are documented, with at least 40% negative or boundary cases.
- Minimum unit and integration test requirements are met for both applications.
- All five SQL validation queries return zero rows.

- All six cross-application test cases pass in both test cycles.
- Every Must requirement has at least one designed test case and one automated test.
- No open Critical or High severity defects remain.
- Both automated test suites pass, except documented failures associated with open defects.
- Required test reports and documentation are completed.

Main Risks

- Python and Java may interpret shared database data differently.
- Incorrect validation may allow invalid expense information.
- Database transaction failures may create inconsistent records.
- Incorrect expense status transitions may allow unauthorized changes.
- Report calculations may produce incorrect totals.
- Limited testing time may delay defect resolution.

Smoke Test Subset

The following critical tests will be executed first to confirm that the main application functionality works:

1. Verify that an employee can log in successfully.
2. Verify that a manager can log in successfully.
3. Verify that an employee can submit a valid expense.
4. Verify that the submitted expense appears in the employee's expense list.
5. Verify that the manager can view the pending expense.
6. Verify that the manager can approve or deny the expense.
7. Verify that the employee can see the updated decision and manager comment.
8. Verify that reports display expense totals correctly.
9. Verify that both applications connect to the same SQLite database.

Smoke test pass criteria: All critical workflows must work without application crashes or unexpected errors.